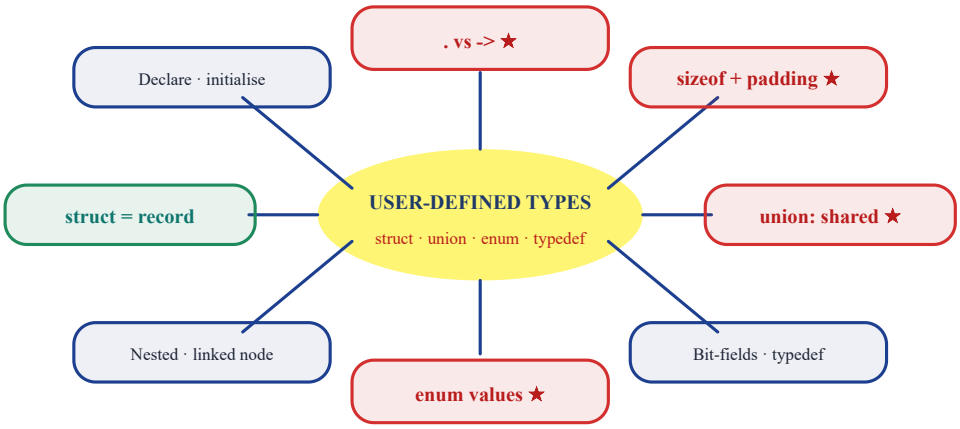
User-Defined Data Types in C

struct · . vs -> · padding · bit-fields · union · enum · typedef: exam-ready in 11 sheets



↑ the whole lesson on one page: red = high-yield, green = the foundation

Syllabus coverage

The C syllabus line: "**User defined data types: structures, unions, enumerations**". Pointers themselves have their own lesson; reading and writing structures to files belongs to the file I/O lesson.

High-yield list

① struct: every member has its own memory
② union: all members share offset 0
③ sizeof(union) = largest, rounded up
④ sizeof(struct) ≥ sum (padding)
⑤ p->x ≡ (*p).x; *p.x ≡ *(p.x)
⑥ s1 = s2 allowed; s1 == s2 error
⑦ enum starts at 0; next = previous + 1
⑧ enum constants are `int`
⑨ no address of a bit-field
⑩ `typedef char *P; P a, b;` → both pointers

Legend: **yellow** = key term · **red circle** = must-memorise · **purple box** = memory trick · **red box** = exam trap · **green** = worked example

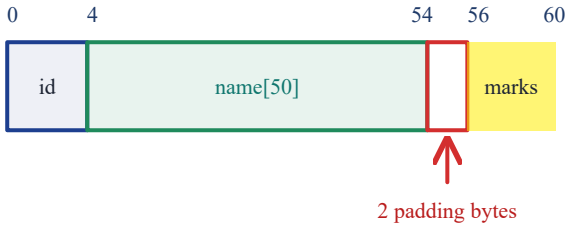
Contents

✓ Structures: declare & initialise … p. 2
✓ Access: . vs ->, functions … p. 3
✓ Arrays, nested & self-referential … p. 4
✓ sizeof, padding & alignment … p. 5
✓ Bit-fields … p. 6
✓ Unions … p. 7
✓ Enumerations … p. 8
✓ typedef + struct vs union vs enum … p. 9
✓ Night-before sheet … p. 10
✓ Practice MCQs + written … p. 11

1 Structures: Declaration, Definition, Initialisation

Definition: A **structure** (`struct`) is a user-defined type that groups related variables, usually of **different** types, under one name. Each variable inside is a **member** (field). Members are stored **in declaration order**, and **each member has its own memory**. Declaring the struct **type** (the template) allocates nothing; defining a **variable** of that type allocates memory.

```c
#include <stdio.h>
#include <string.h>
struct Student {          /* tag */
    int   id;
    char  name[50];
    float marks;
};                        /* ; needed */
int main(void) {
    struct Student s1;
    s1.id = 101;
    strcpy(s1.name, "Ruhul");
    s1.marks = 85.5f;
    printf("%d %s %.2f\n",
        s1.id, s1.name, s1.marks);
    return 0;  /* 101 Ruhul 85.50 */
}
```



4 + 50 + (2) + 4 = 60 bytes

float must start at a multiple of 4

one *struct Student* in memory

Way to create variables	Code	Note
after the type	`struct Student s1, s2;`	keyword `struct` is required in C
with the declaration	`struct Student { … } s1, s2;`	type and variables together
anonymous struct	`struct { int x, y; } pt;`	no tag: no more variables later
positional initialiser	`= {101, "Ruhul", 85.5f}`	in member order
designated initialiser (C99)	`= {.marks = 85.5f, .id = 101}`	any order; name[] becomes ""
partial / zero	`= {101}` · `= {0}`	members not given are **zero**

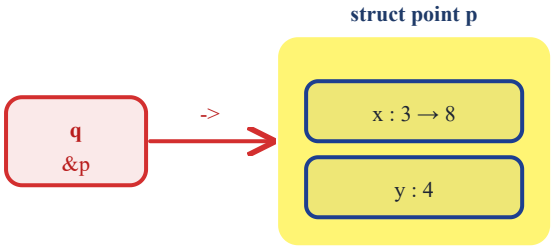
Memory trick: **"The struct is a blank form; a variable is a filled copy."** Printing the blank form costs no paper (no memory); every filled copy does.

Exam trap: `s1.name = "Ruhul";` is a **compile error**: an array cannot be assigned. Use `strcpy()`, or initialise at definition. Forgetting the **;** after **}** is also an error.

Worked example ✍ sizeof: With 4-byte `int`/`float`: 4 + 50 = 54, pad to 56 for *marks*, + 4 → `sizeof(struct Student)` = **60**, not 58.

2 Member Access: . vs ->; Structures and Functions

Definition: The **dot operator** `.` picks a member from a structure **variable**. The **arrow operator** `->` picks a member through a **pointer** to a structure: `p->x` is exactly `(*p).x`. The brackets matter: `.` binds tighter than unary *, so `*p.x` means ***(p.x)**.



p.x == q->x == (*q).x

dot on the box, arrow through the pointer

```
struct point { int x, y; };
struct point p = {3, 4};
struct point *q = &p;
q->x = q->y * 2;
printf("%d %d", p.x, p.y);
/* Output: 8 4 */

void byVal(struct point a) { a.x = 0; }
void byPtr(struct point *a) { a->x = 0; }
/* byVal(p): p unchanged (copy)
   byPtr(&p): p.x becomes 0     */
```

Expression	Means	Expression	Means
`p->x`	`(*p).x`	`++p->x`	`++(p->x)`: member increments
`*p.x`	`*(p.x)`: x must be a pointer	`p++->x`	read x, then p moves on
`&s.x`	`&(s.x)`: member's address	`*p->name`	first char of the name

Operation on whole structures	Allowed?	Why / how
assignment `s2 = s1;`	**yes**	member-by-member copy, arrays inside included (same type only)
comparison `s1 == s2`	**no**: compile error	compare members one by one; `memcmp` may also compare padding
pass to / return from function	yes	by value copies the whole struct; pass a pointer for big structs

Memory trick: "**Dot when you hold the box, arrow when you point at it.**" A variable name → `.`; a pointer → `->`.

Exam trap: Pass-by-value works on a **copy**: after `byVal(p)`, p is unchanged. Also: `::` is C++ only; C has no scope operator for members.

Worked example ✍ return a struct:

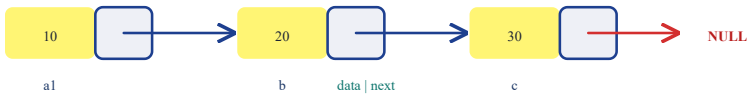
`struct point mid(struct point a, struct point b) { struct point m = {(a.x+b.x)/2, (a.y+b.y)/2}; return m; }` mid({2,4},{6,8}) → **{4, 6}**.

3 Arrays of Structures; Nested and Self-Referential Structures

Definition: An **array of structures** stores many records of one type contiguously: `struct Student st[3];` → `st[0].id`, `st[2].name`. A pointer to the array moves one **whole structure** per `p++` (sizeof the struct, not 1 byte). A **nested structure** has a struct member; a **self-referential structure** has a member that is a **pointer to its own type**.

```
struct pt { int x, y; };   /* 8 B */
struct pt a[3] = {{1,2},{3,4},{5,6}};
struct pt *p = a;      /* &a[0] */
p += 2;                /* a[2]   */
printf("%d %d", p->x,
       (p-1)->y + a[0].x);
/* 5 5;  a at 1000 -> p = 1016 */
struct Date { int d, m, y; };
struct Emp {
    char name[20];
    struct Date dob;   /* nested */
} e, *pe = &e;
e.dob.y = 1995;  pe->dob.m = 6;
```

```
struct node {
    int data;
    struct node *next;  /* OK */
 /* struct node n;  -> error */
};
struct node c = {30, NULL};
struct node b = {20, &c};
struct node a1 = {10, &b};
printf("%d", a1.next->next->data);
/* Output: 30 */
```



three self-referential nodes = a linked list

Kind	Member	Size (4-byte int, 8-byte pointer)	Access
array of structures	`struct pt a[3]`	3 × 8 = 24	`a[i].x`, `(p+i)->x`
nested	`struct Date dob` in Emp	20 + 12 = 32	`e.dob.y`, `pe->dob.y`
self-referential	`struct node *next`	4 + 4 pad + 8 = **16** (8 on 32-bit)	`n->next->data`

Memory trick: "**A box may hold the address of a box like itself, never the box itself.**" A struct containing itself would be infinitely large.

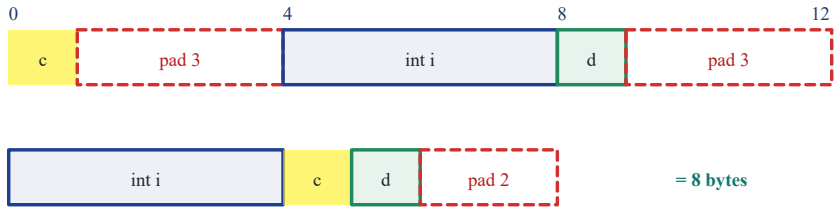
Exam trap: `p++` on `struct pt *` adds **8** bytes (sizeof), not 1. In `e.dob.y` both dots are needed; `pe->dob->y` is wrong because *dob* is not a pointer.

Worked example ✍ walk: `for (p = a; p < a + 3; p++) sum += p->y;` → 2 + 4 + 6 = **12**. (Real lists get nodes from `malloc`: pointers lesson.)

4 sizeof, Padding and Alignment

Definition: The CPU reads data fastest at **aligned** addresses, so the compiler puts each member at an offset that is a **multiple of its alignment** and inserts unused **padding bytes** where needed. Then the whole struct is rounded up to a multiple of its **largest member alignment**, so that arrays of it stay aligned. Hence **sizeof(struct) ≥ sum of members**.

Assumption (typical 64-bit GCC/Clang/MSVC): char 1 · short 2 · int, float 4 · double, long long, pointer 8 — each aligned to its own size.



same three members, 4 bytes saved by reordering

top: {char c; int i; char d;} = 12 · bottom: {int i; char c; char d;} = 8

struct members (in order)	Layout (offsets)	sizeof
`char c; int i; char d;`	c@0, pad 3, i@4, d@8, pad 3	**12**
`int i; char c; char d;`	i@0, c@4, d@5, pad 2	**8**
`char c; double d;`	c@0, pad 7, d@8	16
`char c; short s; int i;`	c@0, pad 1, s@2, i@4	8
`char a; double b; int c;`	a@0, pad 7, b@8, c@16, tail pad 4	24
`double b; int c; char a;`	b@0, c@8, a@12, tail pad 3	16
`int a; char b[5];`	a@0, b@4..8, tail pad 3	12
`char c; int *p;`	c@0, pad 7, p@8 (32-bit: pad 3, p@4)	16 (8)

Memory trick: "**Big rocks first.**" Declare members from largest to smallest alignment (double → int → short → char) and padding nearly disappears.

Exam trap: Padding is **implementation-defined**. On 32-bit Linux (i386) a double inside a struct is 4-aligned, so `{char; double}` = 12 there. If the options have no padded value, the setter assumed **no padding** (plain sum).

Worked example ✍ step by step:

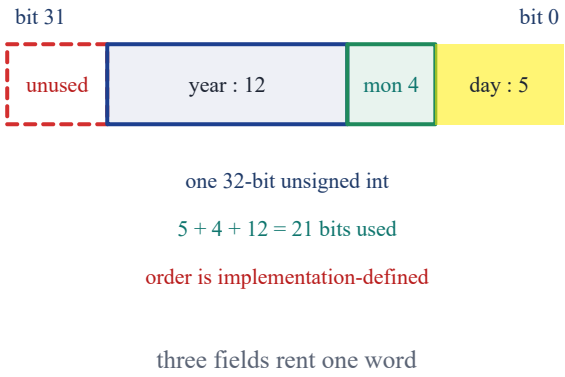
`{char a; double b; int c;}`: 1 → pad to 8 → +8 = 16 → +4 = 20 → round to 8's multiple = **24**.

Packing: `#pragma pack(1)` (or GCC `__attribute__((packed))`) removes padding: `{char c; int i; char d;}` becomes **6** bytes, but unaligned access is slower. Used for file formats and network headers.

5 Bit-Fields

Definition: A **bit-field** is a struct member declared with an explicit width in bits: `type name : width;`. The type must be `int`, `signed int`, `unsigned int` or `_Bool` (compilers may allow others). Adjacent bit-fields are packed into one storage unit (usually an `int`), which saves memory for flags, small counters and hardware registers.

```
struct Date {
    unsigned day   : 5;  /* 0-31   */
    unsigned month : 4;  /* 0-15   */
    unsigned year  : 12; /* 0-4095 */
};          /* 21 bits -> 4 bytes */
struct Date d = {14, 10, 2026};
printf("%u/%u/%u", d.day,
       d.month, d.year);
/* 14/10/2026; 3 ints: 12 B */
struct { unsigned x : 3; } s;
s.x = 9;   /* 9 = 1001b, keeps 001 */
printf("%u", s.x);   /* 1 */
```



Rule	Example
No address: & cannot be applied, so no pointer to it and no `scanf` into it	`&d.day` → **compile error**
No arrays of bit-fields; a bit-field cannot be `static`	`unsigned f[4] : 2;` → error
Width ≤ bits of its type; width 0 only for unnamed fields	`int a : 33;` (32-bit int) → error
Unnamed `: 0` pushes the next field to a new unit	`{unsigned a:4; unsigned :0; unsigned b:4;}` = 8
A field that does not fit the rest of a unit starts a new one (GCC)	`{unsigned a:30; unsigned b:4;}` = 8
Fields that fit together share one unit	`{unsigned a:3, b:5, c:8;}` = 4
Plain `int` field: signed or unsigned is implementation-defined	`int f:1` may hold only 0 and −1

Memory trick: "**Bits rent rooms in a word.**" Fields move into the same int until it is full; a `:0` tenant says "next field, new house".

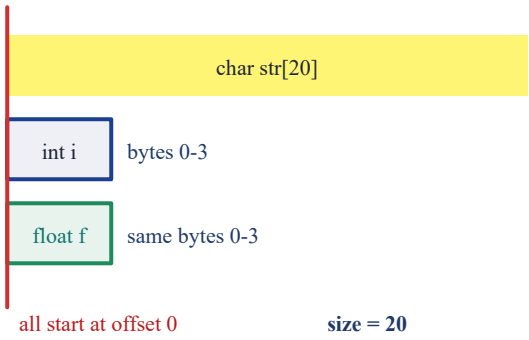
Exam trap: Storing a value too big for an **unsigned** field keeps only the low bits (value mod 2^{width}): 9 in 3 bits → 1. For a **signed** field the result is implementation-defined (GCC: `int a:2; a = 3;` → −1).

Worked example ✍ range: `unsigned m : 4` holds 0 to $2^4 - 1 = \mathbf{15}$; enough for 12 months. A day field needs 5 bits ($31 \le 31$).

6 Unions: One Memory, Many Views

Definition: A **union** is declared like a structure, but **all members start at the same address** (offset 0) and overlap. Its size is the **largest member, rounded up** to the strictest member alignment. Writing one member overwrites the others, so only the **last-written** member holds a meaningful value. Purpose: save memory when only one of several types is needed at a time.

```
union Data {
    int   i;
    float f;
    char  str[20];
};                  /* sizeof = 20 */
union Data d;
d.i = 10;    printf("%d\n", d.i);
d.f = 3.14f; printf("%.2f\n", d.f);
strcpy(d.str, "Hello");
printf("%s\n", d.str);
/* 10 / 3.14 / Hello: each read
   comes right after its write */
d.f = 3.14f;
printf("%d", d.i); /* 1078523331 */
```



three members, one block of memory

Declaration (4-byte int/float, 8-byte double)	Largest member	Alignment	sizeof
`union { int i; float f; char str[20]; }`	20	4	**20**
`union { char c[5]; int i; }`	5	4 → round 5 up	**8**
`union { double d; char c[9]; }`	9	8	16
`struct { int i; float f; char str[20]; }`	— (sum)	4	28

```
/* endianness check */
union { int i; char c[4]; } u;
u.i = 1;
puts(u.c[0] == 1 ? "little-endian"
                 : "big-endian");
/* x86 / ARM default: little */
```

```
/* tagged union */
struct Value {
    enum { INT, FLT } tag;
    union { int i; float f; } v;
};  /* read v.i only if tag == INT */
```

Memory trick: "**Struct = flats in a building; union = one room, many tenants.**" Only the last tenant to move in actually lives there.

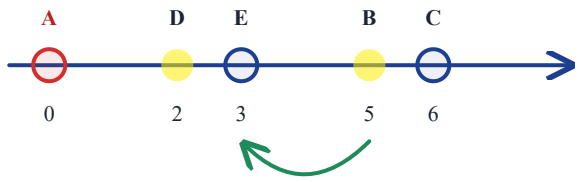
Exam trap: `union Data d = {10};` initialises **only the first member** (i). Use `{.f = 2.5f}` (C99) for another. A union's size is **not** the sum. 1078523331 = 0x4048F5C3, the bit pattern of 3.14f.

Worked example ✍ bytes: `u.i = 0x41424344;` on a little-endian PC, `u.c[0]` is the lowest byte 0x44 → `printf("%c", u.c[0])` prints **D**.

7 Enumerations (enum)

Definition: An **enumeration** is a user-defined type made of **named integer constants** (enumerators). Unless a value is given, the first enumerator is **0** and every later one is **previous + 1**. Enumerators are constants of type `int`; they make code readable (`if (day == Friday)` instead of `if (day == 5)`).

```
enum Week { Sunday, Monday, Tuesday,
   Wednesday, Thursday, Friday,
   Saturday };
enum Week today = Wednesday;
printf("%d", today);       /* 3 */
enum Month { Jan = 1, Feb, Mar, Apr };
                    /* 1 2 3 4 */
enum color { RED, GREEN = 5, BLUE };
printf("%d %d", RED, BLUE); /* 0 6 */
enum { A, B = 5, C, D = 2, E };
       /* 0, 5, 6, 2, 3 */
```



a given value restarts the count

enum { A, B = 5, C, D = 2, E }

Rule	Example / result
Duplicate values allowed	`enum { LOW = 1, MIN = 1 };` ✓
Negative values allowed	`enum { N = -2, M };` → M = −1
Values must be integer constant expressions	`enum { X = 2.5 };` → error; `C = B + 3` ✓
Enumerators share the ordinary identifier namespace	`enum A {X}; enum B {X};` in one scope → **error**
An enumerator is a constant, not a variable	`RED++;` or `RED = 3;` → error
An enum variable is an integer underneath (C)	`enum Week d = 10; d++;` compiles in C (not C++)
`sizeof` an enum variable	implementation-defined, typically **4** (same as int)

Point	`enum`	`#define`
Handled by	compiler (real type, debugger shows names)	preprocessor: blind text replacement
Values	integers only; auto-numbered	anything (float, string); each typed by hand
Scope	obeys block scope	from `#define` to end of file (or `#undef`)

Memory trick: "**Zero to start, plus one to follow, a given value resets the count.**"

Exam trap: After an explicit value the count continues from **it**, not from the position: in `{RED, GREEN = 5, BLUE}` BLUE is 6, not 2.

Worked example ✍️: `enum { P = -1, Q, R = Q + 3, S };` → Q = 0, R = 3, S = **4**.

8 typedef; struct vs union vs enum

Definition: **typedef** gives a new **name** (alias) to an existing type; it creates no new type and no memory. With structures it removes the need to write `struct` every time. The struct **tag** (`struct Student`) and the typedef name (`Student`) live in different namespaces, so the same word can be used for both.

```
typedef struct Student {
    int id;  char name[50];
} Student;          /* tag = alias */
Student s1;         /* no 'struct' */

typedef unsigned char BYTE;
typedef int (*Cmp)(const void *,
                   const void *);
Cmp f = compareInts; /* for qsort */
```

```
typedef char *PTR;
PTR a, b;      /* a, b: char *  */

#define PTR2 char *
PTR2 c, d;     /* char *c, d;
                  d is a char!  */
/* 64-bit: sizeof(b) = 8,
           sizeof(d) = 1        */
```

Point	typedef	#define
Processed by	compiler; follows scope rules	preprocessor; plain text substitution
Pointer alias, two variables	**both pointers**	only the first is a pointer

Feature	struct	union	enum
What it holds	several members together	one member at a time	named int constants
Memory	separate for each member	shared, all at offset 0	a variable = one int
sizeof	sum + padding	largest, rounded up	usually 4
Initialiser	all members in order	first member only (or designated)	one enumerator / int
Access	`.` and `->`	`.` and `->`	name used directly
Typical use	records, list nodes	variants, type punning, saving memory	states, menus, days

What is the output? (64-bit, little-endian)

① copy, not alias: `typedef struct { int x; } T; T a = {5}, b; b = a; b.x++; printf("%d %d", a.x, b.x);` → **5 6** (a copy).

② struct size: `struct S { int a; char b; };` `sizeof(struct S)` → **8** (4 + 1 + 3 tail pad).

Memory trick: "**typedef renames, #define rewrites.**" The compiler understands a typedef; the preprocessor just pastes text.

Exam trap: Without typedef, C needs `struct Student s;` — plain `Student s;` is an error in C (allowed in C++). Writing whole records with `fwrite`: file I/O lesson.

Night-Before-the-Exam Sheet

Fact sheet:

struct: own memory per member

union: all members at offset 0

sizeof(struct) = members + padding

sizeof(union) = largest, rounded up

`p->x` ≡ `(*p).x`

`*p.x` ≡ `*(p.x)`

`s2 = s1` ✓ · `s1 == s2` ✗

union init → first member only

enum: 0, then previous + 1

enum constant type = `int`

bit-field: no `&`, no arrays

unsigned n-bit field: 0 … $2^n - 1$

struct may hold a pointer to itself

`p++` moves sizeof(struct) bytes

Sizes to know (4-byte int, 8-byte double/pointer)

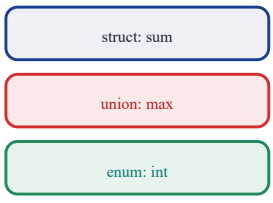
Type	Size	Type	Size
`{char; int; char}`	12	`{int; char; char}`	8
`{char; double}`	16	`{int id; char name[50]; float m;}`	60
`union {int; float; char[20]}`	20	`union {char[5]; int}`	8
`{int data; struct node *next;}`	16	`{unsigned a:3, b:5, c:8;}`	4

Top 10 one-liners for MCQs

1. Struct members are stored in declaration order
2. Declaring a struct type allocates no memory
3. Arrays inside a struct: use `strcpy`, not `=`
4. Designated initialiser: `{.id = 1}` (C99)
5. Pass by value → function edits a copy
6. A union holds one valid member at a time
7. Endianness test: `union {int; char[4]}`
8. Enum duplicates and negatives are legal
9. Same enumerator in two enums → error
10. `typedef char *P; P a, b;` → two pointers

Top traps: sizeof ≠ sum of members · BLUE after GREEN = 5 is 6 · `*p.x` needs brackets · no `==` on structs · `&bitfield` is illegal · plain `Student s;` needs typedef in C · `#define` pointer alias makes one pointer.

All tricks together: blank form vs filled copy · dot when you hold it, arrow when you point · a box holds the address of a box, never itself · big rocks first · bits rent rooms · one room, many tenants · zero to start, plus one to follow · typedef renames, #define rewrites.



three sizes

✓ Practice MCQs (exam pattern, 10 marks)

1. `union { int i; double d; char c[10]; }` with 4-byte int and 8-byte double has size:
(a) 22 (b) 10
(c) 16 (d) 8

2. Which operator reaches a structure member through a pointer?
(a) `.` (b) `->`
(c) * (d) `&`

3. In `enum { X, Y = 4, Z };` the value of Z is:
(a) 2 (b) 4
(c) 6 (d) 5

4. Which operation is NOT allowed on whole structures in C?
(a) comparison with `==` (b) assignment
(c) passing to a function (d) returning from a function

5. `sizeof(struct { char c; int i; char d; })` on a typical 32/64-bit compiler:
(a) 6 (b) 8
(c) 12 (d) 16

6. The address of a bit-field member:
(a) cannot be taken (b) is always even
(c) is the struct's address (d) needs a cast

7. After `typedef char *P; P a, b;` the type of *b* is:
(a) char (b) int
(c) char ** (d) char *

8. A structure in C may contain:
(a) a member of its own type (b) a pointer to its own type
(c) a function body (d) only int members

9. The members of a union:
(a) share one address (b) get separate memory
(c) form a list (d) are all pointers

10. The expression `*p.x` is parsed as:
(a) `(*p).x` (b) `p->x`
(c) `&p.x` (d) `*(p.x)`

Answer key → 1-c · 2-b · 3-d · 4-a · 5-c · 6-a · 7-d · 8-b · 9-a · 10-d

Written question: Distinguish between a structure and a union with an example. For `{int i; float f; char s[10];}` declared once as a struct and once as a union, find both sizes (4-byte int/float). Write an enum for traffic-light states and show its values. 10 marks

1. **Difference (4)** — struct: every member has its own memory, all valid together, size = sum + padding; union: members overlap at offset 0, only the last-written is valid, size = largest rounded up; both use `./->`. 4
2. **Sizes (3)** — struct: 4 + 4 + 10 = 18 → round to multiple of 4 = **20**; union: largest 10 → round to 4 = **12**. 3
3. **enum (3)** — `enum Light { RED, YELLOW, GREEN };` → 0, 1, 2; `enum Light s = GREEN; printf("%d", s);` prints 2; enumerators are int constants. 3

Before the exam ✓

- ✓ Can compute sizeof with padding
- ✓ Can choose `.` or `->` correctly
- ✓ Can trace enum values
- ✓ Can give a union's size and init rule
- ✓ Know the bit-field restrictions
- ✓ Can spot the typedef/#define trap